

TÀI LIỆU ÔN BẢO VỆ LUẬN VĂN

Kiến thức nền — Đào sâu theo thứ tự ưu tiên

CIES: Kỹ thuật xử lý mất cân bằng ảnh hưởng thế nào tới độ ổn định
của giải thích SHAP trong phát hiện gian lận thẻ tín dụng

Biên soạn từ [reports/study_guide.md](#) đối chiếu trực tiếp với 11 file mã nguồn trong [src/](#)
Cập nhật 2026-09-28

Mục lục & cách dùng

Tài liệu chia 3 **tier** theo mức độ bắt buộc phải hiểu sâu khi bảo vệ. Mỗi mục gồm: **Khái niệm** (định nghĩa cốt lõi) → **Cơ chế** (công thức/logic thật) → **Trích code** (nguyên văn từ source, có file + số dòng) → **Vì sao dùng ở đây** → **Câu hỏi phản biện**. Đọc Tier 1 trước — đó là những gì hội đồng gần như chắc chắn hỏi vì nó là "trái tim" của đề tài.

TIER 1 — Bắt buộc hiểu sâu (core mechanism)

- Bootstrap resampling — nền tảng đo độ ổn định
- Quy trình 5 bước của 1 run CIES
- Rank-weighted distance & công thức CIES score
- 4 họ SHAP Explainer: Tree/Linear (exact) vs Deep/Kernel (approx)
- Time-based split & Customer-identity leakage
- PR-AUC vs ROC-AUC trong bài toán cực mất cân bằng

TIER 2 — Nên hiểu vững (cơ chế kỹ thuật của pipeline)

- 5 kỹ thuật xử lý mất cân bằng — cơ chế nội suy & trọng số lớp
- Ablation one-hot phân số (SNAP_SYNTHETIC_ONEHOT)
- Target Encoding out-of-fold + StratifiedGroupKFold (leakage khi bootstrap)
- Optuna: TPE Sampler, MedianPruner, time-series validation folds
- Kiến trúc 5 model & vì sao mỗi model cần xử lý riêng

TIER 3 — Biết cơ chế cơ bản (hạ tầng, ít bị hỏi sâu)

- Cô lập subprocess (spawn vs fork) & xung đột OpenMP
- Ghi file JSON an toàn & cache resample có fingerprint
- Hạ tầng reproducibility & pattern "kind" dispatch

TIER 1 — Bắt buộc hiểu sâu

Đây là cơ chế cốt lõi tạo ra con số CIES. Không hiểu được các mục này thì không thể trả lời bất kỳ câu phản biện nào về "đề tài đo cái gì, đo bằng cách nào".

1 Bootstrap resampling — nền tảng đo độ ổn định

KHÁI NIỆM

Bootstrap là lấy mẫu **có hoàn lại** (with replacement) từ 1 tập dữ liệu gốc, tạo ra 1 "phiên bản" mới cùng cỡ nhưng thành phần dòng hơi khác (một số dòng xuất hiện nhiều lần, một số dòng vắng mặt). Đây là kỹ thuật kinh điển của Efron (1979) để ước lượng độ biến thiên (variance) của 1 estimator khi không có nhiều tập dữ liệu độc lập thật.

CƠ CHẾ TRONG ĐỒ ÁN

Ý tưởng cốt lõi: nếu 1 giải thích SHAP là "ổn định", thì train lại mô hình trên nhiều bản bootstrap khác nhau của cùng 1 tập train phải cho ra **cùng 1 bảng xếp hạng feature**. Bootstrap ở đây đóng vai trò "nguồn nhiễu có kiểm soát" — mô phỏng việc "nếu ta thu thập lại dữ liệu train một chút khác đi thì model có kể lại cùng 1 câu chuyện về feature quan trọng không?".

TRÍCH CODE

```
src/explainability/cies.py#L330-338
```

```
base = df_train.reset_index(drop=True)
sampled = base.sample(frac=1.0, replace=True, random_state=seed)
row_groups = sampled.index.to_numpy()
train_resample = sampled.reset_index(drop=True)
```

`frac=1.0, replace=True` = đúng định nghĩa bootstrap (lấy đủ n dòng, có hoàn lại). `random_state=seed` với `seed = run_idx (0..19)` đảm bảo 20 lần bootstrap khác nhau nhưng tái lập được (reproducible).

Vì sao dùng ở đây: đây là biến độc lập duy nhất bị thay đổi giữa 20 run — encoding, imbalance technique, model, eval set đều cố định. Có lập đúng 1 nguồn nhiễu (dữ liệu train) để CIES chỉ đo phản ứng của giải thích với thay đổi dữ liệu train, không lẫn với nhiễu khác (đối chiếu mục 4 — vì sao không dùng nhiều seed train/test split khác nhau).

Q: Bootstrap có tạo dòng trùng — có ảnh hưởng gì đến các bước sau không?

Có — xem mục 9 (StratifiedGroupKFold): dòng trùng phải bị buộc vào cùng 1 fold khi target-encode, nếu không sẽ rò rỉ nhãn.

2 Quy trình 5 bước của 1 run CIES

KHÁI NIỆM

Mỗi run (trong 20 run) không chỉ là "train lại" — mà lặp lại **toàn bộ pipeline từ đầu** trên dữ liệu bootstrap, để không có bước nào bị "đóng băng" gây thiên lệch giả.

CƠ CHẾ — ĐÚNG 5 BƯỚC, KHÔNG ĐƯỢC RÚT GỌN

```
src/explainability/cies.py#L269 (run_cies_experiment), khớp comment L8-14
```

1. Bootstrap resample train (KHÔNG đổi eval set)
2. Encoding fit lại trên resample này (encode_train, KHÔNG dùng lại encoding cũ)
3. Imbalance handling (apply_imbalance — chỉ trên train đã encode)
4. Train model (build_model + train_model, tham số ĐÃ ĐỒNG BẢNG từ tune.py)
5. SHAP trên tập eval CỐ ĐỊNH (250 mẫu: 50 fraud + 200 hợp lệ, không đổi qua 20 run)

Điểm dễ bị hỏi nhất: nếu bước 2 dùng lại encoding đã fit 1 lần từ đầu (không re-fit mỗi run), thì độ "ổn định" đo được sẽ bị đánh giá **quá cao giả tạo** — vì encoding chính là nguồn dao động (target encoding phụ thuộc thống kê của đúng bản bootstrap đó).

BẢO VỆ CHỐNG CIES GIẢ (SHAP TOÀN 0)

```
src/explainability/cies.py#L394-398
```

```
if np.allclose(shap_vals, 0.0):
    raise ValueError("SHAP toàn 0 — model không học được (dự đoán hằng số), bỏ run này")
```

Nếu model dự đoán hằng số (không học được gì — hay gặp khi 1 tổ hợp bootstrap + kỹ thuật xấu làm train hỏng), mọi feature có SHAP = 0 → mọi thứ hạng hoà nhau → rank-weighted distance = 0 → `cies_score = 1.0`. Đây là "ổn định tuyệt đối" GIẢ (vì không có gì dao động, không phải vì model nhất quán). Guard này loại bỏ run đó khỏi phép đo thay vì để nó kéo điểm CIES lên sai.

Vì sao dùng ở đây: đây chính là điểm phân biệt CIES với việc "chỉ so 2 bảng SHAP là xong" — độ nghiêm ngặt của quy trình (re-encode, guard chống giả) là thứ khiến con số có ý nghĩa khoa học.

3 Rank-weighted distance & công thức CIES score

KHÁI NIỆM

CIES cần 1 hàm đo "khoảng cách" giữa 2 bảng xếp hạng feature (ranking), nhưng không đối xử mọi feature như nhau: đảo hạng ở **feature quan trọng nhất** (rank 1, 2...) phải bị phạt nặng hơn đảo hạng ở feature ít quan trọng (rank 18, 19...) — vì đảo hạng ở top mới thực sự đổi "câu chuyện" mà model kể ra khi giải thích 1 dự đoán.

CƠ CHẾ — CÔNG THỨC ĐẦY ĐỦ

src/explainability/cies.py#L44-83 (compute_rank_weighted_distance)

```
weights = 1.0 / np.minimum(ranks_a, ranks_b) # trọng số theo hạng THẤP NHẤT giữa 2 lần
weights = weights / weights.sum()           # chuẩn hoá tổng = 1
rank_diff = np.abs(ranks_a - ranks_b)
normalized_diff = rank_diff / (n - 1)       # chuẩn hoá về [0, 1]
distance = np.sum(weights * normalized_diff)
```

Điểm tinh tế: trọng số dùng $\min(\text{rank}_a, \text{rank}_b)$ — nghĩa là 1 feature từng lọt top ở **ít nhất 1 trong 2** lần chạy thì thay đổi của nó đã đủ bị phạt nặng, không cần cả 2 lần đều ở top. Bản sửa lỗi ghi rõ trong comment: bản cũ vô tình gán trọng số theo **vị trí cột** (index i) thay vì theo hạng thực tế — làm kết quả phụ thuộc thứ tự cột feature, một bug tinh vi.

TỪ KHOẢNG CÁCH SANG CIES SCORE

src/explainability/cies.py#L148-210 (compute_stability_metric)

```
rankings = [shap_to_ranks(sv) for sv in shap_values_runs] # 20 runs → 20 rankings
# so mọi CẤP trong 20 run: C(20,2) = 190 cặp
for i in range(n_runs):
    for j in range(i+1, n_runs):
        dist = compute_rank_weighted_distance(rankings[i], rankings[j])
        distances.append(dist)
cies_score = 1.0 - mean(distances) # 1 = ổn định tuyệt đối, 0 = hỗn loạn
```

shap_to_ranks (L101-115) xếp hạng feature theo $\text{mean}(|\text{SHAP}|)$ qua các mẫu eval — dùng `scipy.stats.rankdata(method="average")` để feature hoà điểm (vd. cùng SHAP=0) chia đều hạng trung bình, tránh tạo chênh lệch giả chỉ do thứ tự cột (comment L90-94, cùng loại bug như trên).

MA TRẬN ĐỒNG THUẬN (KHÁC CIES, DỄ NHẦM)

src/explainability/cies.py#L118-145 (compute_condition_agreement_matrix)

Dùng CÙNG công thức rank-weighted distance nhưng so sánh **giữa các điều kiện khác nhau** (vd. 5 kỹ thuật imbalance của cùng 1 model) — khác CIES gốc là so **giữa các lần bootstrap của CÙNG 1 điều kiện**. Nếu bị hỏi "CIES và ma trận đồng thuận khác nhau ở đâu" — đây chính là câu trả lời.

Vì sao thiết kế vậy: nếu chỉ dùng Spearman correlation thông thường (đối xử mọi hạng như nhau), sẽ không phân biệt được trường hợp "feature quan trọng nhất bị đảo với feature quan trọng nhì" (ngghiêm trọng) và "feature hạng 17 đảo với hạng 18" (không đáng kể) — 2 trường hợp cho Spearman gần như bằng nhau nhưng ý nghĩa thực tế rất khác.

Q: Vì sao cần cả Spearman lẫn CIES, không chỉ 1 chỉ số?
Spearman (L192-196, cùng hàm compute_stability_metric) đối chiếu độc lập — nếu 2 chỉ số lệch nhau nhiều ở 1 trường hợp cụ thể, đó là dấu hiệu đáng ngờ cần kiểm tra lại (feature bằng 0 hàng loạt, outlier...). Đây là 1 dạng "sanity check" giữa 2 phép đo cùng mục đích nhưng khác trọng số.

4 họ SHAP Explainer: exact vs approximate

KHÁI NIỆM

SHAP (Shapley Additive exPlanations) gán mỗi feature 1 "đóng góp" vào dự đoán, dựa trên lý thuyết trò chơi hợp tác (Shapley value — trung bình đóng góp biên qua mọi hoán vị thứ tự feature). Tính chính xác (exact) đòi hỏi duyệt 2^n tổ hợp feature — không khả thi với nhiều feature, nên mỗi loại model có 1 thuật toán xấp xỉ/rút gọn riêng.

CƠ CHẾ — MAPPING BẮT BUỘC THEO MODEL

src/explainability/shap_utils.py#L23-33 (EXPLAINER_MAP)

Model	Explainer	Loại	Vì sao
Logistic Regression	LinearExplainer	Exact	công thức đóng cho model tuyến tính
RF / XGBoost / CatBoost	TreeExplainer	Exact	khai thác cấu trúc cây (Lundberg 2020)
ANN	DeepExplainer	Approx, tất định	dựa trên DeepLIFT
ANN (fallback)	KernelExplainer	Approx, có nhiễu	Monte Carlo sampling

VÌ SAO KHÔNG DÙNG KERNEL CHO ANN — ĐIỂM HAY BỊ HỎI NHẤT MỤC NÀY

src/explainability/shap_utils.py#L28-31 (comment)

"DeepExplainer thay KernelExplainer: do trên ANN thật, Kernel chạy lặp 2 lần cùng model chỉ khớp nhau Spearman ~0.85 (nhiều lấy mẫu riêng — sẽ làm điểm CIES của ANN thấp GIẢ TẠO vì CIES đo đúng độ dao động thứ hạng), chậm hơn ~5x, còn Deep tất định, cộng dồn đúng (sai số 0) và cùng đơn vị log-odds với LR/XGBoost."

Đây là lập luận quan trọng nhất của cả đề tài về phương pháp luận: KernelExplainer xấp xỉ Shapley value bằng lấy mẫu ngẫu nhiên (weighted linear regression trên các tổ hợp feature con) — nên **2 lần gọi trên cùng 1 model, cùng 1 input** vẫn cho kết quả hơi khác nhau (chỉ tự khớp Spearman ~0.85 với chính nó). Nếu dùng Kernel cho ANN, phần "bất ổn" đo được sẽ trộn lẫn giữa: (a) nhiễu thật do kỹ thuật imbalance/bootstrap — thứ CIES muốn đo, và (b) nhiễu giả do chính explainer — thứ không liên quan gì đến câu hỏi nghiên cứu. DeepExplainer tất định (deterministic) loại bỏ hẳn nguồn nhiễu (b).

TRÍCH CODE — LOGIC TỰ LUI (FALLBACK) KHI DEEPEXPLAINER LỖI

src/explainability/shap_utils.py#L226-234

```
try:
    explainer = shap.DeepExplainer(ann, X_bg_tensor)
    shap_values = explainer.shap_values(X_eval_tensor)
except Exception as e:
    logger.warning(f"DeepExplainer lỗi ({type(e).__name__}: {e}) — lùi về KernelExplainer.")
    return _compute_shap_kernel(model, X_eval, X_background), "kernel"
```

Tên explainer **thực sự đã dùng** được ghi lại vào run_logs mỗi run (ciesz.py#L413 : "explainer": explainer_used) — để biết nếu có run nào lùi về Kernel thì phải diễn giải điểm CIES của nó thận trọng hơn. Guide xác nhận "cả 200 run ANN đều dùng Deep" — nghĩa là fallback này chưa từng kích hoạt trong benchmark thật.

CHI TIẾT ĐỂ BỎ SÓT: SHAP GIẢI THÍCH TRONG KHÔNG GIAN NÀO?

shap_utils.py#L86-101 (_compute_shap_linear), L206-209 (docstring _compute_shap_deep)

LR: SHAP tính trên input đã **StandardScaler** (cùng scaler lúc train) — nếu quên bước này, LinearExplainer đọc hệ số hồi quy (fit trên không gian scale) nhưng nhận input ở không gian gốc → SHAP values sai hoàn toàn (comment L92-95 nhấn mạnh rủi ro này). ANN: SHAP giải thích **logit** (đầu ra trước sigmoid) — cùng thang log-odds với LR/XGBoost, giúp so sánh xuyên model nhất quán hơn.

Vì sao dùng ở đây: cả đề tài xoay quanh việc so sánh độ ổn định SHAP giữa các model khác nhau — nếu 1 model dùng explainer có nhiễu ngẫu nhiên riêng còn model khác dùng explainer tất định, kết luận "model X ổn định hơn model Y" có thể chỉ phản ánh explainer, không phải bản chất model.

Q: TreeExplainer là exact, DeepExplainer là approximate — điều này có làm lệch so sánh CIES giữa cây và ANN không?
Đây là 1 confound thật (đã nêu ở lần trả lời trước) — không có câu trả lời "đã giải quyết" trong code, cần thừa nhận thẳng đây là giới hạn khi so CIES giữa model cây và ANN.

5 Time-based split & Customer-identity leakage

KHÁI NIỆM

Data leakage là khi thông tin không có sẵn tại "thời điểm dự đoán thật" vô tình lọt vào train theo cách làm hiệu năng đo được cao giả tạo. Có 2 dạng riêng biệt trong đồ án này, dễ bị hỏi lẫn lộn: **temporal leakage** (rò rỉ tương lai) và **identity leakage** (rò rỉ danh tính).

CƠ CHẾ 1 — TEMPORAL LEAKAGE

```
src/data/preprocess.py#L28 (sort_values theo TIMESTAMP_COL)
```

```
data = df.sort_values(TIMESTAMP_COL, kind="mergesort").reset_index(drop=True)
```

Nếu chia random, các dòng của **cùng 1 đợt hack thẻ** (fraud thường xảy ra theo chuỗi/burst) rơi vào cả train lẫn test → model "nhớ" pattern của đúng đợt hack đó thay vì học pattern tổng quát. Số đo thật: chia ngẫu nhiên XGBoost PR-AUC 0,932 → chia theo thời gian 0,882 (mục 4, câu 1 trong guide) — chênh 0,05 chỉ do cách chia, không do model.

CƠ CHẾ 2 — IDENTITY LEAKAGE (TINH VI HƠN, HAY BỊ BỎ SÓT)

```
src/config.py#L58-63
```

```
CUSTOMER_IDENTITY_COLS = ["lat", "long", "city", "state", "job", "city_pop",
                           "merch_lat", "merch_long"]
# 98,6% cặp (lat, long) và 90,1% giá trị city_pop chỉ thuộc 1 thẻ,
# 92,2% city chỉ có 1 thẻ; merch_lat/long luôn nằm trong ~1,4° quanh nhà khách.
```

Khác leakage thời gian: các cột này **trông giống feature hợp lệ** (toạ độ, nghề nghiệp) nhưng vì mỗi giá trị (hoặc tổ hợp) gần như chỉ ánh xạ tới đúng 1 khách hàng, model học được "khách nào từng bị hack thẻ trong quá khứ" thay vì "giao dịch trông như thế nào thì đáng ngờ". Số đo minh chứng rõ nhất (guide mục 4, câu 2): giữ các cột này + chia theo thời gian → PR-AUC chỉ **0,240** (model không khái quát được — vì khách bị hack ở test thường KHÁC khách bị hack ở train, khác thời điểm); bỏ đi → **0,882**.

KIỂM CHỨNG "BỎ KHÔNG MẤT TÍN HIỆU THẬT"

Guide ghi: khoảng cách khách hàng–cửa hàng của giao dịch fraud và hợp lệ gần như trùng nhau (76,3 km so với 76,1 km) — nghĩa là "khoảng cách xa nhà" không thực sự phân biệt fraud/legit; phần PR-AUC 0,24→0,88 mất đi khi bỏ cột không phải vì mất tín hiệu thật, mà vì mất khả năng "tra cứu ngược danh tính".

Vì sao dùng ở đây: đây là 2 lớp phòng thủ khác nhau chống lại 2 cơ chế ảo tưởng hiệu năng khác nhau — nếu chỉ làm 1 trong 2 (chỉ chia theo thời gian mà giữ cột định danh, hoặc ngược lại), PR-AUC vẫn bị thổi phồng.

Q: Sao không phát hiện leakage bằng cách khác (vd. feature importance bất thường)?

Đây chính là cách đã dùng — quan sát PR-AUC bất thường cao (0,932 hay đi kèm cột định danh) là dấu hiệu đầu tiên gợi ý leakage, sau đó mới định lượng bằng tỷ lệ % cặp giá trị chỉ thuộc 1 khách.

6 PR-AUC vs ROC-AUC trong bài toán cực mất cân bằng

KHÁI NIỆM

ROC-AUC: diện tích dưới đường cong True Positive Rate (Recall) vs False Positive Rate. **PR-AUC** (= Average Precision): diện tích dưới đường cong Precision vs Recall. Cả 2 đều threshold-independent (không phụ thuộc 1 ngưỡng cụ thể) nhưng phản ứng rất khác nhau khi lớp dương cực hiếm.

CƠ CHẾ TOÁN HỌC — VÌ SAO ROC-AUC "PHÒNG ĐẠI"

$FPR = FP / (FP + TN)$. Khi TN cực lớn (Sparkov test: 555.719 dòng, chỉ 2.145 fraud → TN có thể lên tới ~553k), FP dù tăng gấp đôi/gấp ba vẫn là 1 số rất nhỏ so với TN → FPR gần như không đổi → ROC-AUC gần như không phân biệt được model tốt/tệ (guide: FDB báo ROC-AUC 0,995–0,998 cho mọi model, không đủ độ phân giải). Ngược lại $Precision = TP / (TP + FP)$ nhạy với FP **tuyệt đối**, bất kể TN lớn cỡ nào.

TRÍCH CODE

```
src/evaluation/metrics.py#L49-53
```

```
metrics["pr_auc"] = average_precision_score(y_true, y_proba) # METRIC CHÍNH
metrics["roc_auc"] = roc_auc_score(y_true, y_proba)          # chỉ tham khảo
```

`average_precision_score` = xấp xỉ diện tích dưới đường cong PR bằng tổng có trọng số của Precision tại mỗi ngưỡng recall thực đạt được — không nội suy tuyến tính giữa 2 điểm (khác cách tính AUC hình thang thông thường), tránh đánh giá quá lạc quan ở đoạn cong lõm.

Vì sao dùng ở đây: Saito & Rehmsmeier (2015) — nguồn được trích trong guide — chứng minh cụ thể ROC-AUC gây hiểu lầm khi lớp dương <1% và khuyến nghị PR-AUC làm chỉ số quyết định. Để tài dùng PR-AUC không chỉ để "chọn model tốt hơn" mà còn vì **PR-AUC ít phụ thuộc kỹ thuật resample** hơn F1 (mục 4 câu 10): resample/class-weighting đổi việc *hiệu chỉnh xác suất* (calibration) nhưng ít đổi *thứ hạng* dự đoán (ranking) — PR-AUC dựa trên ranking nên ổn định hơn F1 (dựa trên 1 ngưỡng cố định 0,5, nhạy với calibration).

TIER 2 — Nên hiểu vững

Cơ chế kỹ thuật đứng sau từng bước của pipeline. Hội đồng có thể hỏi "vì sao chọn cách này" — cần hiểu đủ để giải thích, không cần thuộc lòng từng dòng code.

7 5 kỹ thuật xử lý mất cân bằng — cơ chế thật

KHÁI NIỆM CHUNG

Chia 2 nhóm hoàn toàn khác cơ chế: **4 kỹ thuật resampling** (SMOTE, SMOTE-ENN, ADASYN, Borderline-SMOTE) sinh/xoá dòng dữ liệu; **class weighting** không đổi dữ liệu, chỉ đổi *trọng số trong hàm mất mát* lúc train.

CƠ CHẾ — RESAMPLING FAMILY

[src/imbalance/resamplers.py#L48-54](#) (`get_resampler`)

```
resamplers = {
    "smote": SMOTE(random_state=seed),
    "smote_enn": SMOTEENN(random_state=seed),
    "adasyn": ADASYN(random_state=seed),
    "borderline_smote": BorderlineSMOTE(random_state=seed),
    "class_weighting": None,
}
```

- **SMOTE**: với mỗi điểm minority, chọn ngẫu nhiên 1 trong k-nearest-neighbor cùng lớp, nội suy tuyến tính (điểm giả = điểm gốc + $t \times (\text{hàng xóm} - \text{điểm gốc})$, $t \in [0,1]$) → luôn cân bằng về 1:1.
- **ADASYN**: như SMOTE nhưng sinh *nhiều điểm giả hơn* ở vùng minority khó phân loại (density càng thấp/khó, sinh càng nhiều) — thích ứng theo độ khó cục bộ.
- **Borderline-SMOTE**: chỉ nội suy quanh các điểm minority nằm *sát ranh giới quyết định* (borderline — được xác định qua tỷ lệ hàng xóm khác lớp), bỏ qua các điểm minority nằm sâu trong vùng an toàn.
- **SMOTE-ENN**: SMOTE trước, sau đó Edited Nearest Neighbor xoá các điểm (ở cả 2 lớp) bị đa số hàng xóm khác lớp bao quanh — "dọn nhiễu" quanh ranh giới sau khi oversample.

CƠ CHẾ — CLASS WEIGHTING (KHÔNG SINH DÒNG MỚI)

[src/imbalance/resamplers.py#L65-79](#) (`get_class_weights`)

```
weights = compute_class_weight("balanced", classes=classes, y=y)
# công thức sklearn: w_c = n_samples / (n_classes * n_samples_c)
```

Trên Sparkov (~0,58% fraud): trọng số lớp fraud ≈ 86 , lớp hợp lệ $\approx 0,5$ — nghĩa là 1 lỗi phân loại sai 1 giao dịch fraud bị phạt nặng gấp ~172 lần so với phân loại sai 1 giao dịch hợp lệ. **Áp dụng vào model khác nhau theo cơ chế khác nhau** (điểm hay bị hỏi):

Model	Cách nhận trọng số	File#Line
LR / RF	<code>class_weight={0: w0, 1: w1}</code> trực tiếp vào sklearn	train.py#L166,185
XGBoost	<code>scale_pos_weight = w1/w0</code> (tỷ lệ, không phải dict)	train.py#L211-215
CatBoost	<code>class_weights=[w0, w1]</code> (list theo thứ tự lớp)	train.py#L237-240
ANN	<code>pos_weight</code> trong BCEWithLogitsLoss	train.py#L358-366

Bug lịch sử được ghi lại trong comment (train.py#L206-210) rất đáng nhớ để trả lời "đã kiểm tra kỹ chưa": `scale_pos_weight` từng bị tính **ngược** ($w_0/w_1 \approx 0,0017$ thay vì đúng ≈ 577), khiến lớp fraud bị GIẢM trọng số thay vì tăng — PR-AUC của XGBoost+class_weighting trên ULB tụt từ 0,877 xuống 0,704 do lỗi 1 phép chia bị đảo ngược.

Vì sao dùng ở đây: việc mỗi model nhận "cùng 1 khái niệm trọng số lớp" theo giao diện API khác nhau là lý do `get_class_weights()` chỉ trả về 1 dict chuẩn hoá `{0: w0, 1: w1}` và để `build_model()` tự chuyển đổi cho đúng từng thư viện — tránh lặp logic tính toán rải rác 4 nơi.

8 Ablation one-hot phân số (SNAP_SYNTHETIC_ONEHOT)

KHÁI NIỆM

SMOTE/ADASYN/Borderline-SMOTE nội suy **tuyến tính trên mọi cột**, kể cả cột one-hot (gender, category) — vốn chỉ nên nhận giá trị 0 hoặc 1. Nội suy giữa 2 dòng có category khác nhau tạo ra giá trị phân số vô nghĩa (vd. 0,7/0,3), điều không bao giờ xảy ra ở dữ liệu thật.

CƠ CHẾ — 2 CHẾ ĐỘ, ĐO CẢ 2

src/config.py#L87-98, src/imbalance/resamplers.py#L98-112 (snap_onehot_groups)

```
SNAP_SYNTHETIC_ONEHOT = False # mặc định: SMOTE thuần (one-hot phân số), như literature
# True: ép mỗi nhóm one-hot của MỌI dòng về đúng 1 category (argmax) —
# dòng thật vốn đã hợp lệ nên no-op, chỉ dòng tổng hợp bị thay đổi.
```

Kỹ thuật xử lý rất khéo: thay vì phải biết dòng nào là "tổng hợp" (khó, vì SMOTE-ENN còn xoá bớt dòng làm mất ranh giới), hàm áp argmax cho **mọi dòng** — dòng thật vốn one-hot hợp lệ sẽ không đổi gì (no-op), chỉ dòng có giá trị phân số mới bị "chụp" về 1 category.

KẾT QUẢ ABLATION (ĐÃ ĐO, DÙNG ĐỂ TRẢ LỜI CÂU HỎI 4.7 TRONG GUIDE)

Chế độ	PR-AUC (XGBoost+SMOTE)	CIES
False (one-hot phân số, mặc định)	0,887	0,957
True (ép hợp lệ)	0,766	0,959

Diễn giải: giá trị phân số vô tình **cô lập dòng tổng hợp khỏi vùng dữ liệu thật** (một dạng feature "đánh dấu ngầm" dòng nào là synthetic) — giúp model dễ tách biên hơn (PR-AUC test cao hơn), dù về mặt lý thuyết đó không phải tín hiệu hợp lệ. CIES gần như không đổi giữa 2 chế độ vì độ ổn định xếp hạng feature không phụ thuộc nhiều vào chi tiết này.

Vì sao mặc định False: để so sánh được với cách làm phổ biến trong literature (SMOTE chuẩn, không có bước "làm sạch" one-hot) — nếu literature khác không làm bước này mà đề tài lại làm, so sánh benchmark chéo bài báo sẽ khập khiễng.

9 Target Encoding out-of-fold & leakage khi bootstrap

KHÁI NIỆM

Target encoding thay 1 category (vd. merchant) bằng thống kê của target ứng với category đó (vd. tỷ lệ fraud trung bình của merchant này). Rủi ro cố hữu: nếu dùng thống kê tính từ *chính dòng đang encode*, nhãn của dòng đó "rò" vào feature của chính nó → model học thuộc thay vì tổng quát hoá.

CƠ CHẾ — OUT-OF-FOLD (OOF) + SMOOTHING

src/data/encoding.py#L77-90 (stratified_kfold_target_encode)

```
for train_idx, val_idx in splits:          # StratifiedKFold, 5 fold
    fold_train = df_reset.iloc[train_idx]
    fold_mean = fold_train[target_col].mean() # prior CHỈ từ fold train
    stats = fold_train.groupby(col)[target_col].agg(["mean", "count"])
    smoothed = (stats["count"]*stats["mean"] + smoothing*fold_mean) / (stats["count"]+smoothing)
    encoded.iloc[val_idx] = df_reset[col].iloc[val_idx].map(smoothed).fillna(fold_mean)
```

Mỗi dòng validation được encode bằng thống kê tính từ 4 fold còn lại (không chứa chính nó) — đây là "out-of-fold". **Smoothing** (hệ số 10): category có ít dòng bị kéo về `fold_mean` — tránh category hiếm (vd. merchant chỉ xuất hiện 1-2 lần) nhận giá trị encode cực đoan (0 hoặc 1) chỉ vì trùng hợp ngẫu nhiên với vài quan sát ít ỏi.

TÌNH CHỈNH ĐÃ SỬA (ĐÁNG CHÚ Ý KHI TRẢ LỜI "ĐÃ KIỂM TRA RÒ RỈ CHƯA")

encoding.py#L79-81 (comment)

```
"Prior làm mịn tính CHỈ từ fold train. Dùng trung bình toàn bộ train (bản cũ...)
thì nhãn của chính các dòng validation lọt vào prior — rò rỉ nhỏ nhưng có thật."
```

Đây là ví dụ tốt để trả lời "bạn có tự tìm ra giới hạn/lỗi nào trong quá trình làm không" — đề tài từng có leakage nhỏ (prior tính từ toàn train thay vì chỉ fold train) và đã tự phát hiện, sửa.

VẤN ĐỀ RIÊNG KHI KẾT HỢP VỚI BOOTSTRAP: STRATIFIEDGROUPKFOLD

encoding.py#L52-56 (docstring), L64-73

Bootstrap tạo dòng trùng (1 dòng gốc có thể xuất hiện 3 lần trong 1 bản resample). Nếu dùng `StratifiedKFold` thường, 3 bản sao có thể rơi vào 3 fold khác nhau — khi 1 bản sao ở fold validation, nhãn của 2 bản sao kia (ở fold train) *chính là nhãn của dòng đó* → rò rỉ trực tiếp. Giải pháp: truyền `groups = row_groups` (id dòng gốc) để dùng `StratifiedGroupKFold` — ép mọi bản sao của cùng 1 dòng gốc luôn nằm **cùng 1 fold**.

```
skf = StratifiedGroupKFold(n_splits=n_splits, shuffle=True, random_state=random_state)
splits = skf.split(df_reset, df_reset[target_col], groups=np.asarray(groups))
```

Vì sao dùng ở đây: đây là ví dụ cụ thể của "leakage bậc 2" — 1 kỹ thuật chống leakage (OOF) tự nó cần thêm 1 lớp bảo vệ khác khi kết hợp với 1 kỹ thuật khác (bootstrap). Rất hay bị hỏi vì thể hiện độ hiểu sâu về tương tác giữa các thành phần pipeline, không chỉ hiểu từng phần riêng lẻ.

10 Optuna: TPE Sampler, MedianPruner, time-series folds

KHÁI NIỆM

Hyperparameter tuning tự động — thay vì grid/random search (thử ngẫu nhiên, không học từ lịch sử), **TPE (Tree-structured Parzen Estimator)** là 1 thuật toán Bayesian optimization: mô hình hoá phân phối xác suất của tham số dẫn đến kết quả tốt vs kết quả xấu, rồi đề xuất điểm thử tiếp theo có khả năng tốt cao dựa trên phân phối đó.

CƠ CHẾ — VALIDATION THEO THỜI GIAN, KHÔNG PHẢI K-FOLD THƯỜNG

src/models/tune.py#L94-104 (time_series_folds)

```
splitter = TimeSeriesSplit(n_splits=n_splits, test_size=n_samples // (3 * n_splits))
# 1/3 CỬI train chia thành n_splits khối liên tiếp;
# mỗi fold train trên MỌI dòng đứng TRƯỚC khối validation của nó (cửa sổ mở rộng - expanding window)
```

Đây là lý do y hệt mục 5 (Time-based split) áp lại ở tầng tune: nếu dùng StratifiedKFold ngẫu nhiên để chọn tham số, tham số "học thuộc" tốt nhất sẽ thắng thay vì tham số dự đoán tương lai tốt nhất — 1 dạng leakage khác, lần này ở tầng chọn hyperparameter chứ không phải tầng đánh giá cuối.

CƠ CHẾ — ENCODE RIÊNG TỪNG FOLD (KHÔNG DÙNG CHUNG 1 LẦN ENCODE)

tune.py#L107-140 (encode_time_folds)

```
enc_tr, maps = encode_train(part_tr, ...) # fit CHỈ trên phần train của fold này
enc_val = encode_test(part_val, maps, ...) # transform validation bằng mapping đó
```

Nếu encode 1 lần trên toàn bộ train rồi mới cắt thành 3 fold, target encoding của các dòng ở fold train sẽ đã "nhìn thấy" nhãn của giai đoạn validation phía sau (vì mapping ban đầu tính trên toàn tập, gồm cả tương lai) — nhìn trước tương lai (look-ahead bias), một biến thể khác của temporal leakage.

CƠ CHẾ — MEDIANPRUNER (TIẾT KIỆM COMPUTE)

tune.py#L218-232

```
if use_pruning:
    trial.report(float(np.mean(val_pr_aucs)), fold_idx) # báo cáo TẠM THỜI sau mỗi fold
    if trial.should_prune():
        raise optuna.TrialPruned()
pruner = optuna.pruners.MedianPruner(n_startup_trials=n_startup_trials, n_warmup_steps=0)
```

Sau mỗi fold (không phải sau khi chạy hết cả 3 fold), so PR-AUC trung bình tạm thời với median các trial **đã hoàn thành trước đó** — nếu kém rõ rệt, dừng ngay, không lãng phí compute chạy tiếp fold 2, 3 của 1 trial rõ ràng tệ. n_startup_trials đảm bảo vài trial đầu luôn chạy đủ để có tham chiếu median trước khi bắt đầu prune.

ĐÓNG BĂNG THAM SỐ — DECOUPLED BASELINE PRE-TUNING

tune.py#L4-9 (module docstring)

Tune chạy **trước** và **trên dữ liệu chưa xử lý mất cân bằng**, rồi **đóng băng** (freeze) 1 bộ tham số duy nhất dùng chung cho cả 5 kỹ thuật. Đây là lựa chọn thiết kế có đánh đổi rõ ràng (guide mục 4 câu 4): đảm bảo chênh lệch CIES/PR-AUC giữa 5 kỹ thuật là do bản thân kỹ thuật, không lẫn với việc mỗi kỹ thuật vô tình được tune tham số khác nhau (confounding). Nhược điểm thẳng thắn: tham số không tối ưu riêng cho từng kỹ thuật.

Vì sao dùng ở đây: toàn bộ thiết kế mục này phục vụ đúng 1 nguyên tắc "controlled experiment" — cô lập biến độc lập (kỹ thuật imbalance) khỏi mọi biến gây nhiễu khác (độ phức tạp model tối ưu riêng, leakage thời gian ở tầng tune).

11 Kiến trúc 5 model & vì sao mỗi model cần xử lý riêng

KHÁI NIỆM

5 model đại diện 3 họ khác nhau về bản chất toán học: tuyến tính (LR), cây quyết định tổ hợp (RF, XGBoost, CatBoost), mạng nơ-ron (ANN) — mỗi họ có yêu cầu tiền xử lý và cách nhận trọng số lớp khác nhau, đã nêu ở mục 7.

VÌ SAO CÂY KHÔNG CẦN SCALE NHƯNG LR/ANN CẦN

[train.py#L171-174](#) (comment), [L347-351](#) (comment trong `_train_ann`)

```
# LR: "feature chưa scale (amt, lat/long, hour, target-encode probability...)
# chênh lệch scale lớn khiến lbfgs khó hội tụ trong max_iter (ConvergenceWarning)."
# ANN: "Không scale thì lớp Linear đầu nhận giá trị khổng lồ và train gần như hỏng
# — đo trên Sparkov: PR-AUC 0.16 (không scale) vs 0.74 (có scale)."
```

Cây quyết định chỉ so sánh `feature <= threshold` tại mỗi node — bất biến (invariant) với mọi phép biến đổi đơn điệu (bao gồm scaling), nên không cần `StandardScaler`. LR (gradient-based solver lbfgs) và ANN (gradient descent) đều nhạy với thang đo — chênh lệch scale giữa các feature (vd. `unix_time ~1e9` so với `target-encoded ~1e-2`) làm bề mặt loss méo mó, hội tụ chậm/hỏng.

KIẾN TRÚC ANN CỤ THỂ

[train.py#L42-63](#)

```
Input(21 hoặc 29) → Linear(128) → ReLU → BatchNorm1d → Dropout(0.3)
                  → Linear(64)  → ReLU → BatchNorm1d → Dropout(0.3)
                  → Linear(32)  → ReLU
                  → Linear(1)                                     # logit, KHÔNG có Sigmoid
```

Không có Sigmoid ở cuối vì dùng `BCEWithLogitsLoss` (kết hợp Sigmoid + Binary Cross-Entropy trong 1 phép tính, ổn định số học hơn tính riêng — tránh `log(0)`). `predict_proba` mới tự áp `torch.sigmoid` ([train.py#L433-434](#)) khi cần xác suất thật.

RÀNG BUỘC CATBOOST — KHÔNG DÙNG CƠ CHẾ NỘI TẠI

[train.py#L4-9](#), [L223](#), [tune.py#L11-12,67](#)

```
# "CatBoost PHẢI dùng chung bộ feature đã encode sẵn — KHÔNG dùng cơ chế
# Ordered Target Statistics nội tại (KHÔNG truyền cat_features). Nếu CatBoost
# dùng encoding riêng, sự khác biệt CIES giữa CatBoost và các model khác
# không tách bạch được — phá vỡ RQ3."
```

CatBoost có cơ chế encode categorical riêng rất mạnh (Ordered Target Statistics) — nếu bật, CatBoost sẽ nhận feature khác hẳn 4 model kia, và bất kỳ chênh lệch CIES nào giữa CatBoost và các model khác sẽ không biết là do "bản chất model" hay do "encoding khác nhau" — phá vỡ tính so sánh công bằng (Research Question 3 của đề tài, theo comment).

Vì sao dùng ở đây: nguyên tắc xuyên suốt: mọi khác biệt giữa 5 model phải đến từ *bản thân thuật toán học*, không phải từ việc mỗi model vô tình nhận input/tiền xử lý khác nhau — cùng tinh thần "controlled experiment" như mục 10.

TIER 3 — Biết cơ chế cơ bản

Hạ tầng kỹ thuật hỗ trợ chạy benchmark quy mô lớn. Ít khả năng hội đồng hỏi sâu, nhưng nên biết đủ để không bối rối nếu bị hỏi "vì sao code phức tạp vậy" hoặc "chạy trong bao lâu".

12 Cô lập subprocess (spawn) & xung đột OpenMP

KHÁI NIỆM

OpenMP là runtime chạy song song đa luồng dùng bởi nhiều thư viện số học (torch, xgboost đều dùng bản libomp riêng). Khi 2 thư viện có bản libomp khác nhau cùng nạp vào 1 process, chúng có thể xung đột (2 runtime cùng quản lý 1 thread pool) → segfault hoặc treo vô thời hạn.

CƠ CHẾ — LAZY IMPORT + SUBPROCESS SPAWN

src/models/train.py#L18-28 (comment), src/utils/isolation.py#L16-26

```
# torch chỉ import khi build ANN; xgboost/catboost chỉ import khi build đúng model đó
# → 1 subprocess chỉ bao giờ nạp ĐÚNG 1 thư viện model, không phụ thuộc thứ tự vòng lặp.

ctx = mp.get_context("spawn") # KHÔNG dùng "fork" (mặc định Linux)
# "fork" nhân bản NGUYÊN TRẠNG memory của process cha — nếu cha đã lỡ nạp
# torch/xgboost, con vẫn kế thừa trạng thái đó, không giải quyết được gì.
# "spawn" khởi động 1 Python interpreter HOÀN TOÀN MỚI mỗi lần.
```

CƠ CHẾ — TIMEOUT VỚI ESCALATION SIGTERM → SIGKILL

isolation.py#L49-72 (_terminate_hard)

```
process.terminate() # SIGTERM trước
process.join(timeout=KILL_GRACE_SECONDS)
if process.is_alive():
    process.kill() # SIGKILL nếu SIGTERM không đủ
    process.join(timeout=KILL_GRACE_SECONDS)
```

Comment giải thích lý do cụ thể (không phải phòng xa lý thuyết): process.join() không có timeout mặc định — nếu process con treo trong 1 syscall không ngắt được bằng SIGTERM (vd. GPU driver deadlock ở tầng kernel), riêng terminate() không đủ, **đã tái hiện thật** 1 job Optuna trên Kaggle "Running" hơn 5 giờ dù đặt timeout=3600s.

Vì sao dùng ở đây: benchmark có 25 tổ hợp (Sparkov) × nhiều lần chạy, nếu 1 tổ hợp treo mà không tách subprocess, toàn bộ notebook (và kết quả các tổ hợp đã chạy xong nhưng chưa lưu) sẽ mất theo.

13 Ghi file JSON an toàn & cache resample có fingerprint

KHÁI NIỆM

Nhiều tiến trình (script CIES chạy song song, hoặc merge kết quả từ Kaggle) có thể cùng ghi vào 1 file kết quả JSON — nếu không kiểm soát, tiến trình ghi sau sẽ đè mất phần tiến trình khác vừa thêm.

CƠ CHẾ — KHOÁ ĐỘC QUYỀN + GHI ATOMIC

src/utils/jsonio.py#L17-56 (update_json)

```
with _exclusive_lock(path):
    current = json.load(f) if path.exists() else default()
    new = update(current) # callback: đọc → sửa → trả về bản mới
    tmp = path.with_name(path.name + ".tmp")
    json.dump(new, ...) # ghi ra file TẠM trước
    os.replace(tmp, path) # đổi tên = ghi atomic (không bao giờ có file dở dang)
```

Pattern read-modify-write trong 1 khoá độc quyền (mutex) đảm bảo không tiến trình nào đọc phải bản ghi dở, và os.replace (đổi tên trong cùng filesystem) là 1 phép toán nguyên tử ở tầng OS — không có trạng thái trung gian "file nửa ghi xong".

CƠ CHẾ — CACHE RESAMPLE CÓ FINGERPRINT (SHA1)

src/imbalance/resamplers.py#L158-165 (_resample_fingerprint)

```
for part in (technique, seed, SNAP_SYNTHETIC_ONEHOT, onehot_groups,
             imblearn.__version__, sklearn.__version__, X.shape, X.dtype, y.dtype):
    h.update(repr(part).encode())
h.update(X.tobytes()); h.update(y.tobytes())
```

Kết quả resample (vd. SMOTE-ENN trên toàn train mất ~2,5 giờ) được cache thành file .parquet, nhưng chỉ dùng lại nếu "dấu vân tay" (hash của toàn bộ input + cấu hình + version thư viện) khớp — dữ liệu, kỹ thuật, seed, hoặc version thư viện resample đổi (thuật toán lảng giềng gần có thể thay đổi hành vi giữa các version) đều làm fingerprint khác đi → tính lại, không bao giờ dùng nhầm cache cũ.

Vì sao dùng ở đây: benchmark 5 model × 5 kỹ thuật cần phép resample giống hệt nhau cho mọi model (cùng technique) — cache tránh tính lại đúng 1 phép SMOTE-ENN 5 lần (1 lần/model) một cách lãng phí.

14 Hạ tầng reproducibility & pattern "kind" dispatch

KHÁI NIỆM

Reproducibility (tái lập được) nghĩa là chạy lại đúng cùng code + cùng cấu hình phải ra cùng kết quả — điều kiện tiên quyết để 1 thí nghiệm khoa học đáng tin.

CƠ CHẾ

src/config.py#L9-11

```
SEED = 42
N_RUNS = 20 # số lần lặp CIES - tối thiểu 20 (guide: có thể tăng 30 nếu cần)
```

SEED lan truyền xuống mọi nơi có tính ngẫu nhiên (resampler, model, Optuna sampler, bootstrap) — nhưng lưu ý: mỗi trong 20 run CIES dùng seed = run_idx (0..19) khác nhau *có chủ đích* (để 20 lần bootstrap thực sự khác nhau), còn benchmark 1-lần (train_and_evaluate_combo) thì seed cố định = 42 cho mọi tổ hợp (để so sánh công bằng giữa các kỹ thuật).

PATTERN "KIND" — TRÁNH ĐOÁN NGẦM KIỂU MODEL

train.py#L104-126 (model_kind)

```
def model_kind(model):
    if not isinstance(model, dict): return "other"      # sklearn/xgb/catboost
    kind = model.get("kind")
    if kind not in ("lr", "ann"):
        raise ValueError(f"... phải được tạo từ build_model(), Keys hiện có: {...}")
    return kind
```

Comment giải thích rõ động cơ thiết kế: LR và ANN cần bọc thêm state (scaler, epochs...) nên trả về dict thay vì object model thuần — nhưng "6 chỗ lặp lại cùng suy luận ngầm" (đoán qua việc dict có key "scaler" hay không) từng là nguồn lỗi tiềm ẩn nếu thêm key mới trùng tên. Giải pháp: gắn nhãn "kind" tường minh ngay lúc tạo, buộc raise lỗi rõ ràng nếu dict không đến từ build_model() — thà lỗi sớm và rõ ràng còn hơn suy luận sai âm thầm.

Vì sao đáng nhắc trong bảo vệ: đây là ví dụ nhỏ nhưng thể hiện tư duy kỹ thuật phần mềm tốt — 1 refactor phòng ngừa lỗi trước khi nó xảy ra, không phải chỉ chạy được là xong.

Nguồn: Toàn bộ trích dẫn code trong tài liệu này lấy trực tiếp từ 11 file source (đã đọc nguyên văn ngày 2026-09-28): [config.py](#), [data/preprocess.py](#), [data/encoding.py](#), [imbalance/resamplers.py](#), [models/train.py](#), [models/tune.py](#), [evaluation/metrics.py](#), [explainability/shap_utils.py](#), [explainability/cies.py](#), [utils/isolation.py](#), [utils/jsonio.py](#), đối chiếu với [reports/study_guide.md](#). Khi ôn, mở song song file source tương ứng để xem toàn bộ ngữ cảnh xung quanh đoạn trích.